



Конспект «Индексы»

1. EXPLAIN — демонстрирует этапы выполнения запроса и может быть использован для оптимизации.

```
EXPLAIN FORMAT = TRADITIONAL
SELECT *
FROM customer c
JOIN address a ON a.address_id = c.address_id
JOIN city c2 ON c2.city_id = a.city_id
WHERE c2.city_id = 17;
```

2. EXPLAIN ANALYZE — производит EXPLAIN вывод вместе с синхронизацией и дополнительной, основанной на итераторах, информацией о том, как ожидания оптимизатора совпадают с фактическим выполнением.

```
-> Nested loop inner join (cost=1.68 rows=1) (actual
time=0.088..0.092 rows=1
loops=1)

  -> Index lookup on a using idx_fk_city_id (city_id=17)
(cost=0.95 rows=1)
(actual time=0.070..0.071 rows=1 loops=1)

    -> Index lookup on c using idx_fk_address_id
(address_id=a.address_id)
(cost=0.72 rows=1) (actual time=0.016..0.018 rows=1 loops=1)
```

3. Индексы — инструмент, который позволяет оптимизировать выборку из базы данных, значительно сокращая время на получение данных.

- Создадим временную таблицу и внесем в нее данные:

```
CREATE TABLE film_temp (  
    film_id INT,  
    title VARCHAR(50),  
    description TEXT,  
    language_id INT,  
    release_year INT  
);  
INSERT INTO film_temp  
SELECT film_id, title, description, language_id,  
release_year FROM film;
```

- Посмотрим на план запроса, где нужно получить фильм с id = 100:

```
EXPLAIN  
SELECT *  
FROM film_temp  
WHERE film_id = 100;
```

- Добавим на столбец film_id ограничения первичного ключа, которое включает индекс:

```
ALTER TABLE film_temp ADD PRIMARY KEY (film_id);
```

- Изменим немного данные и получим фильмы по двойному условию:

```
UPDATE film_temp  
SET release_year = 2005  
WHERE film_id <= 500;
```

- Создадим составной индекс (COMPOSITE KEY):

```
CREATE INDEX lang_year ON film_temp(language_id,  
release_year);
```

- Проверим, что индексы есть:

```
SELECT *  
FROM INFORMATION_SCHEMA.STATISTICS  
WHERE TABLE_NAME='film_temp';
```

- Получим фильмы по двойному условию с составным индексом:

```
EXPLAIN  
SELECT *  
FROM film_temp  
WHERE language_id = 1 AND release_year = 2006;
```

4. Уникальные индексы

```
ALTER TABLE `имя_таблицы` ADD UNIQUE INDEX `имя_индекса` ...;  
CREATE UNIQUE INDEX `имя_индекса` ...;
```

5. Стоимость индексов

- Если в операциях **SELECT** индексы ускоряют работу, то в операциях **INSERT** и **UPDATE** время увеличивается, как и занимаемое место на жестком диске. Удалим индексы во временной таблице **film_temp**:

```
ALTER TABLE film_temp DROP PRIMARY KEY;  
DROP INDEX lang_year ON film_temp;
```

- Посмотрим на размер таблицы без индексов:

```
SELECT table_name, data_length, index_length  
FROM INFORMATION_SCHEMA.TABLES  
WHERE table_name = "film_temp";
```

- Добавим индекс **PRIMARY KEY** и составной индекс **lang_year** и проверим размер:

```
ALTER TABLE film_temp ADD PRIMARY KEY (film_id);  
CREATE INDEX lang_year ON film_temp(language_id,  
release_year);  
SELECT table_name, data_length, index_length  
FROM INFORMATION_SCHEMA.TABLES  
WHERE table_name = "film_temp";
```